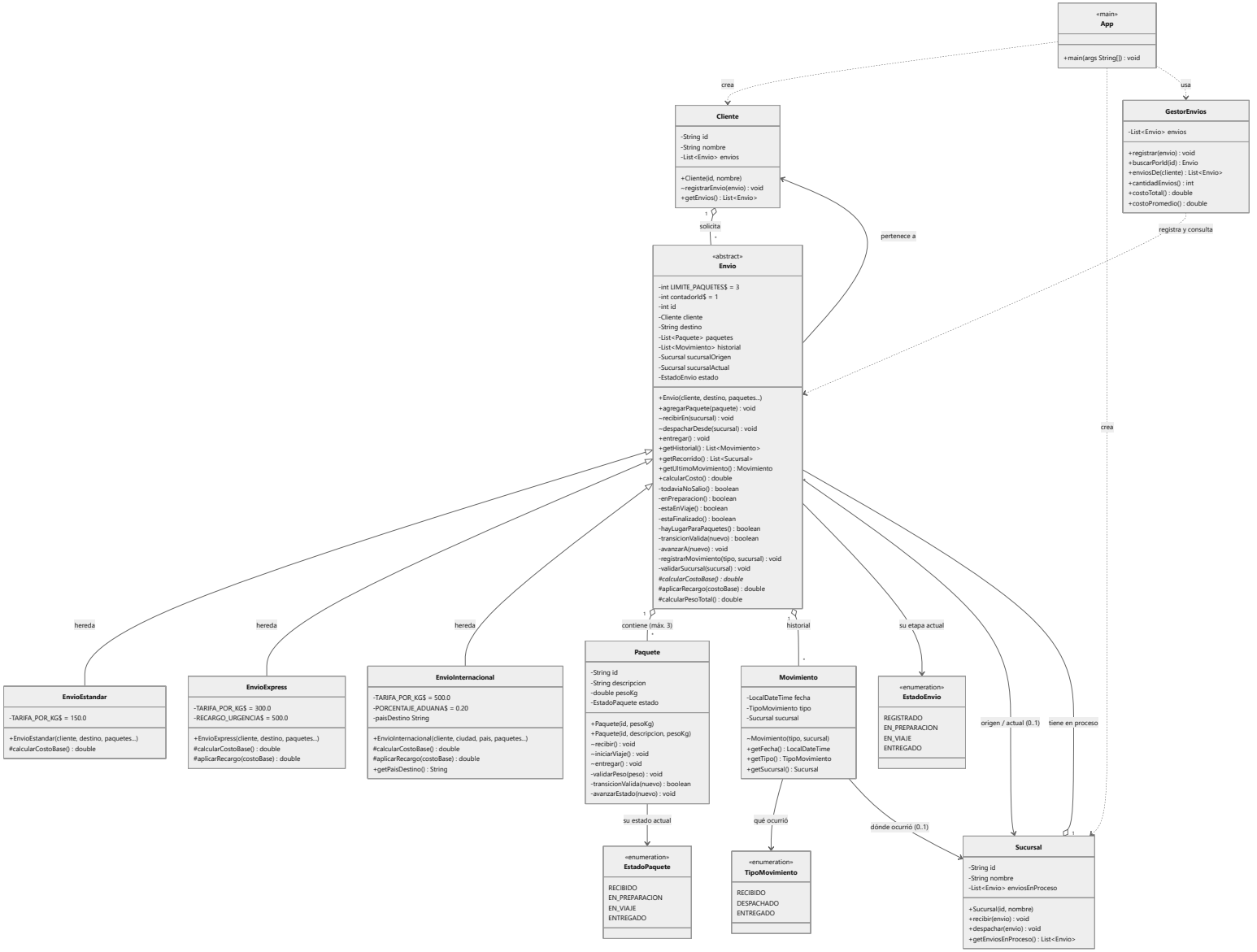
Diagrama de Clases

TP1 — Desarrollo de Software · Sistema de Gestión de Envíos y Distribución



Leyenda de notación

Símbolo	Significado
`-`	Atributo o método `private`
`+`	Método `public`
`#`	Método `protected`
`~`	Método `package-private` (visible solo dentro del sistema)
`*` al final	Método **abstracto**, debe implementarse en las subclases
`$`	Miembro `static`, compartido por todas las instancias
`<<enumeration>>`	Enumeración de valores fijos
`<<abstract>>`	Clase que no se puede instanciar directamente
`<|--`	Herencia: la subclase *es un* tipo de la superclase
`o--`	Asociación: un envío *contiene* paquetes
`..>`	Dependencia: una clase *crea o usa* a otra

Los *getters* simples (`getId()`, `getDestino()`, `getPesoKg()`, …) se omiten por claridad; todos existen en el código fuente.

Ciclo de vida del paquete

RECIBIDO	→ `recibir()` →	**EN_PREPARACION**	→ `iniciarViaje()` →	**EN_VIAJE**	→ `entregar()` →	**ENTREGADO**

Cada etapa solo avanza de a un paso. Saltarse una genera `IllegalStateException`. Esos métodos son `package-private`: los invoca únicamente el `Envio` del que depende el paquete.

Ciclo de vida del envío

REGISTRADO	→ `recibirEn()` →	**EN_PREPARACION**	→ `despacharDesde()` →	**EN_VIAJE**	→ `entregar()` →	**ENTREGADO**

Las sucursales mueven al envío: `Sucursal.recibir(envio)` · `Sucursal.despachar(envio)`; la entrega es del propio envío porque el destino final no siempre es una sucursal. Una sucursal intermedia recibe o despacha un envío `EN_VIAJE` sin cambiar su etapa: solo registra el paso en el historial.